Advancing Literature Review Automation through API-Based Searches: Design of an Independent Emulator

Completed Research Paper

Anna Geßler
Universität Hamburg
Bundesstraße 56a, 20146 Hamburg
anna.gessler@studium.uni-hamburg.de

Karl Schnickmann
Otto-Friedrich-Universität Bamberg
An der Weberei 5, 96047 Bamberg
k.schnickmann@stud.uni-bamberg.de

Gerit Wagner
Frankfurt School of Finance & Management
Adickesallee 32-34, 60322 Frankfurt am Main
g.wagner@fs.de

Abstract

The automation of literature search execution is becoming increasingly important as publication volumes grow and AI-based, agentic systems augment academic search activities. However, some Application Programming Interfaces (APIs) lack support for academic search queries consisting of nested subqueries and Boolean operators. This constraint complicates systematic and reproducible searches, increases manual effort, and decreases search efficiency. To resolve this issue, we present the Query Emulation for Academic Literature Searches (QuEALS) technique, which allows researchers to conduct automated API-based searches with academic search queries even when APIs do not support them natively. This approach works by recursively processing the query, retrieving literature from the API, and filtering the results for subqueries. We followed a Design Science Research approach to design, implement, and evaluate QuEALS. The prototype was implemented and evaluated for Crossref, which offers one of the most comprehensive and open research APIs, but lacks support for academic search queries.

Keywords: Literature review, automation, APIs, design science research.

Introduction

The automation of literature searches increasingly depends on APIs that can execute academic search queries reliably and reproducibly. Growing publication volumes have intensified the challenge of efficiently identifying relevant research (Larsen et al., 2019), while the Information Systems discourse increasingly considers AI-based and agentic approaches to augmenting and automating literature reviews (e.g., Wagner et al., 2022; Wagner et al., 2026). Because literature reviews commonly rely on systematic database searches, efficient and reproducible execution of search queries is a central requirement (Gusenbauer & Haddaway, 2021; Hiebl, 2023; Velez-Estevez et al., 2023). In increasingly automated workflows, this execution depends critically on Application Programming Interfaces (APIs): local literature review software, web-based tools, and AI-based agents rely on APIs to submit queries, retrieve records, and integrate search results into subsequent processing steps (e.g., Wagner & Prester, 2024; Wang et al., 2026; Zwanenburg, 2023). Consequently, the capabilities of academic search APIs determine which forms of literature-search automation are feasible in practice.

Many academic search APIs cannot execute the nested Boolean queries required for systematic literature searches. Such queries typically combine search terms and fields using Boolean operators such as *AND*, *OR*, and *NOT*, often within nested subqueries (MacFarlane et al., 2022). Yet insufficient support for these query structures remains a major shortcoming of prominent search systems (Gusenbauer & Haddaway, 2020) and is also found in APIs such as *Crossref*, *CiteSeerX*, and *DBLP* (Li & Rainer, 2022). This limitation is particularly consequential for *Crossref*: its large, cross-disciplinary corpus and openly reusable metadata make it an especially promising foundation for literature searches because, unlike many proprietary search systems, retrieved metadata can be openly reported, shared, and reused without comparable redistribution restrictions. This creates important opportunities for transparent and reproducible literature-search workflows that its limited query functionality currently constrains.

Existing work has largely treated missing query capabilities as shortcomings of academic search systems to be addressed by database owners and search-system providers (Gusenbauer & Haddaway, 2020, 2021; Li & Rainer, 2022). However, despite persistent demand for richer query functionality, these capabilities have remained unimplemented. To the best of our knowledge, prior software for academic literature searches does not provide an independent query-processing layer that emulates capabilities missing from the underlying search system. We therefore adopt a Design Science approach to develop an artifact that independently emulates nested Boolean *AND* and *OR* operations for APIs that do not support them natively. Accordingly, our research question is: *How can limitations of APIs, such as the lack of full nested Boolean operator support, be overcome without relying on the cooperation of search system providers?*

Following a Design Science Research approach (Hevner et al., 2004; Peffers et al., 2007), we design and evaluate the Query Emulation for Academic Literature Searches (QuEALS) technique. QuEALS provides an independent, deterministic query-processing layer that emulates nested AND and OR operations when these capabilities are not supported natively by an API. We implement and evaluate the technique for the Crossref API. QuEALS thereby contributes a mechanism for extending API-based literature-search execution without requiring changes to the underlying search system. Rather than replacing AI-based or agentic approaches, it complements them by providing reliable execution of specified query semantics within automated and AI-augmented search workflows.

Foundations of Academic Literature Searches

Academic literature searches can be categorized as lookup, exploratory, or systematic searches (Gusenbauer & Haddaway, 2021). While lookup searches retrieve specific information, exploratory searches support broader investigation of a research area, and systematic searches follow predefined strategies to enable comprehensive, transparent, and reproducible retrieval. Our work primarily applies to exploratory and systematic searches. These searches may employ journal-driven, database-driven, seminal-work-driven, or combined techniques (Hiebl, 2023). Among these, database searches are widely used and typically rely on keywords and structured search queries to retrieve literature from academic search systems (Al-Zubidy & Carver, 2019; Prester et al., 2021). Accordingly, we focus on database searches and on the capabilities required for their systematic and increasingly automated execution.

Database Searches

Database searches comprise three phases: planning, conducting, and reporting (Gusenbauer & Gauster, 2025), which provide a higher-level structure for more detailed search activities (Cooper et al., 2018). Planning concerns the development of the search strategy, conducting its operational execution, and reporting its transparent and reproducible documentation. Our work focuses specifically on query execution within the conducting phase: translating a specified search strategy into database requests and retrieving the corresponding records.

Database queries typically combine keywords, search fields, Boolean or proximity operators, and parentheses for nested structures (Al-Zubidy & Carver, 2019). A common formulation strategy is the Building-Blocks approach, in which synonymous terms within concepts are connected by OR, while different concepts are combined with AND (Booth, 2008). Such queries can become structurally complex and may require systematic variation during search design.

Recent Information Systems research has developed software to support this query formulation and variation. For example, *permusearch* (Zwanenburg, 2023) generates permutations of search terms and scopes together with their expected yields, while *LitBaskets* (Wang et al., 2026) examines query yields across different journal scopes. These approaches illustrate how software can support the systematic exploration of alternative search strategies. They do not, however, address limitations in the subsequent execution of specified queries by the underlying search systems, which is the focus of our work.

Modalities of Database Search Systems

Database searches can be conducted through web interfaces, web crawlers or scrapers, and APIs, which offer different levels of automation. Web interfaces require substantial manual interaction, while crawlers and scrapers automate parts of this process but may be restricted by search providers and are vulnerable to interface changes (Gonzalez-Toral et al., 2019; Li & Rainer, 2022; Shakeel et al., 2018). API-based searches instead retrieve literature programmatically and provide a stronger basis for integrating search execution into automated review workflows (Haddaway et al., 2022; Shakeel et al., 2018; Velez-Estevez et al., 2023). However, their query functionality remains constrained by the capabilities of the underlying source. For example, *Publish or Perish* enables searches across multiple academic data sources but inherits their respective query limitations. QuEALS differs by implementing missing query-processing functionality outside the underlying search system.

The Role of Independent Emulators in Future Search Infrastructures

The need for independent emulators arises when scholarly APIs lack support for nested query structures or particular operators (Boolean or proximity). Several important APIs, including *Crossref* and *DBLP*, lack support for nested Boolean queries (Gusenbauer & Haddaway, 2020; Li & Rainer, 2022; Velez-Estevez et al., 2023), thereby preventing their use for many systematic search strategies. An instructive survey of Gusenbauer & Haddaway (2020) shows that issues with Boolean queries are a key limitation across contemporary search systems. In line with *Cochrane*, *The Campbell Collaboration*, and *CEE* for systematic reviews, Gusenbauer & Haddaway (2020) highlight that support for (nested) Boolean queries is a necessary capability for search systems in the context of academic literature reviews. Of the 28 search systems evaluated, 14 did not fully support Boolean queries, with APIs particularly affected. These persistent limitations create an opportunity for independent query emulators that compensate for missing query-processing capabilities outside the academic search system.

With our work, we challenge the prevalent tacit assumption that deficiencies in scholarly search systems can only be addressed by the providers of these systems. Prior research has frequently adopted such a provider-centric perspective, identifying missing capabilities as deficiencies of the underlying platforms and directing calls for improvement toward database owners and search-system providers (Gusenbauer & Haddaway, 2020, 2021; Li & Rainer, 2022). This makes improvements dependent on providers' priorities and willingness to modify their systems, even when important requirements remain unimplemented for extended periods, as illustrated by the persistent lack of nested Boolean support in *Crossref*. Independent emulators reduce dependency on provider-implemented search functionality by moving selected query-processing capabilities beyond the platform boundary and implementing them in external components. They do not, however, eliminate dependency on the underlying retrieval infrastructure: access to records remains subject to API availability, rate limits, metadata consistency, and other provider-side constraints. Thus, independence in this context refers to query processing rather than independence from the underlying search system. This opens up a broader design space for literature search methods and infrastructures, in which new capabilities can be developed independently of the underlying platform providers.

As such, independent emulators for academic search queries can enable a range of usage scenarios. First, researchers can use emulators directly through local programming or graphical interfaces. Second, third-party software, such as cloud-based literature review platforms, could integrate the same emulator modules to provide support for academic search queries through existing interfaces. Finally, agentic AI systems may incorporate emulators in the form of command-line-based skills. More generally, activities across the planning, conducting, and reporting phases of database searches may be delegated according to the capabilities they require. During planning, LLM-based agents may be particularly suitable for activities that benefit from broad and creative language processing, such as identifying or refining synonyms, while

domain experts remain important where substantive knowledge and scholarly judgment are required (Wagner et al., 2026). By contrast, executing an already specified search strategy requires reliable and deterministic implementation of its query semantics. While AI agents may, in principle, access APIs directly and attempt to emulate queries, independent emulator modules provide deterministic logic, can implement a reproducible procedure, and may require fewer computational resources at runtime. Independent emulators and agentic systems can therefore serve complementary roles across the search process, with agents supporting planning and workflow orchestration, and deterministic components executing queries when the underlying API lacks the required functionality. In conclusion, emulators can serve as an independent enhancement of existing search infrastructure that provides reusable search capabilities to researchers, review software, and agentic AI systems.

Methodology

This paper followed the Design Science Research methodology, as outlined by Peffers et al. (2007) and Hevner et al. (2004). The design of the QuEALS technique was driven by a problem-centered approach, focusing on addressing specific challenges in the automation of academic literature searches, particularly the lack of API support for nested Boolean queries. In developing QuEALS as the central artifact, we followed the Design Science Research process outlined by Peffers et al. (2007), iterating between design, development, and evaluation.

Problem Identification and Motivation

As outlined in the introduction, there is a significant gap between what researchers need from academic search systems and what is already available (Li & Rainer, 2022). APIs lack many features that would be needed for fully featured APIs. Due to these limitations in the APIs, the execution of database searches cannot be fully automated yet. Solving this problem focuses on extending the functionality of API-based searches in search systems such as *Crossref* that lack native Boolean query support. This new approach allows researchers to compensate for shortcomings of existing search systems by emulating the required capabilities independently.

Objectives of the Solution

Our objective is to enhance the process of academic literature searches by designing and implementing the QuEALS technique. The ultimate goal is to make literature searches more efficient, allowing researchers to automate tasks that currently rely on manual actions like retrieving literature through a web interface. A key focus of this work is to develop a prototype that implements the QuEALS technique and thereby demonstrates how local query emulation can extend the functionality of existing APIs. By extending APIs with enhanced search capabilities and enabling more complex search queries, the need for manual workarounds through web interfaces and local post-filtering is reduced.

The formulation of search queries involves various search capabilities, including (1) Boolean operators, (2) nested queries, (3) phrase searching (4) proximity operators, (5) (multi-)field searching and (6) truncation and wildcard-based term expansions (Gusenbauer & Haddaway, 2020; MacDonald et al., 2024). Given the importance of Boolean searching in systematic search strategies, the Boolean operators AND and OR were selected as the initial focus of QuEALS. These Boolean operators enable researchers to combine search concepts through nested queries, thereby supporting the retrieval of relevant records (Gusenbauer & Haddaway, 2020). The emulation of AND and OR is particularly important for *Crossref*, as it does not natively support Boolean operators, which limits its functionality for conducting complex searches through its web interface or API.

The QuEALS prototype currently addresses the first three capabilities – Boolean operators, nested queries, and phrase searching – by emulating nested Boolean operators and applying exact term and phrase matching. Other search capabilities remain outside the current implementation but are considered as potential areas for future work. By emulating the Boolean operators AND and OR, QuEALS aims to enhance API-based query execution for *Crossref* and demonstrates a technique for extending APIs with limited native functionality to support more complex academic literature search queries.

Design and Development

The artifact was designed in multiple design-evaluation cycles. A detailed description of the artifact's design is provided in section 4. The prototype was implemented in Python and demonstrated on the *Crossref* API. Before implementing the artifact, different search systems and their shortcomings were scanned to select the search system for which the prototype would be implemented. The following section explains the selection of *Crossref* for the demonstration and evaluation.

Demonstration

After considering several search systems, including *DBLP*, *Google Scholar*, *Crossref*, and *Semantic Scholar*, we selected *Crossref* to demonstrate the prototype. The selection was based on three criteria: support for basic keyword searches, lack of support for nested Boolean queries, and availability of result-count estimates, which enable a more efficient design. While *DBLP* and *Semantic Scholar* also meet these criteria, they already support individual AND and OR operators; *Crossref* therefore provides a stronger test case for demonstrating that QuEALS can operate on APIs limited to simple keyword searches. *Crossref* is also particularly suitable because its metadata is cross-disciplinary (Velez-Estevez et al., 2023), it provides access to a large corpus of DOI-registered records, and requests for Boolean-query support have remained unresolved for many years[1][2].

Evaluation

The evaluation of the prototype focuses on assessing its correctness, retrieval completeness and performance. We selected four queries to test the prototype (see Table 1) and measure the occurrence of errors, as well as runtime. The first error type (false positives) refers to records that are returned by QuEALS but do not match the search query. These errors are considered when evaluating correctness (see Correctness Evaluation). We define correctness as the absence of false positives in the result set, which, in information retrieval terminology, corresponds to a precision rate of 100%. To ensure that the prototype worked correctly, we used a custom query selection method to verify that all returned records match the input query.

The second error type (false negatives) refers to records in *Crossref* that match the query, but that are not retrieved by QuEALS. We define retrieval completeness as the absence of false negatives in the result set, which, in information retrieval terminology, corresponds to a recall rate of 100%. To evaluate retrieval completeness, a comparison was performed between the results obtained with QuEALS and those from a journal search (see Retrieval Completeness Evaluation). We assessed retrieval completeness by comparing result sets from the manual journal search with result sets from QuEALS to ensure that no relevant records are missing.

Finally, we evaluated the runtime performance of the prototype in real-world settings. For each of the four queries, we measured the time from starting the prototype until the prototype terminated and returned the results. We identify API constraints on record retrieval, such as rate limits, as the primary performance bottleneck and examine the relationship between retrieval volume and runtime. We discuss search filters as a practical strategy for improving prototype feasibility and performance. To assess their potential time savings, we estimate the yields of queries with journal and/or time filters and compare them with those of unfiltered searches, demonstrating that such filters can substantially reduce the number of records retrieved and, consequently, runtime. These filtered queries reflect common scenarios in academic research, where the search scope is restricted to specific publication periods or journals.

Communication

The prototype solution, its design process and the problem that motivated it are described in detail in this paper. Beyond presenting the solution as a design science research project, we communicated the solution to the public in the following ways: Firstly, the prototype code is integrated in a Python library and made

[1] https://community.crossref.org/t/crossref-api-questions-test-queries-example/999

[2] https://community.crossref.org/t/advanced-set-boolean-operators/4596

openly available (Eckhardt et al., 2026). Secondly, we responded to various posts on the *Crossref* forum which requested support for Boolean queries and referred them to our solution.

Artifact Design

This section presents the design of the QuEALS artifact. The QuEALS artifact and prototype were developed in multiple design-evaluation cycles to fulfill all defined objectives and to achieve a high-quality design outcome.

Design

This section describes the design and functionality of the artifact. The core concept of the QuEALS artifact and the main algorithm are illustrated in Figure 1. The goal of the QuEALS artifact is to add crucial missing functionality—particularly support for Boolean operators—to the search process without relying on the database provider. This is achieved by independent emulation of missing features, either locally or as part of third-party services. As a result, the artifact acts as an intermediary component between the search system and the user.

In a conventional search, the user manually executes a query via the database provider's web interface. QuEALS automates the execution of queries by retrieving records via an API and filtering them to return exactly the set of results that matches the input query. The artifact requires only access to a simple keyword search via the API to retrieve records; the actual query processing and filtering functionality are decoupled from the database provider's search system.

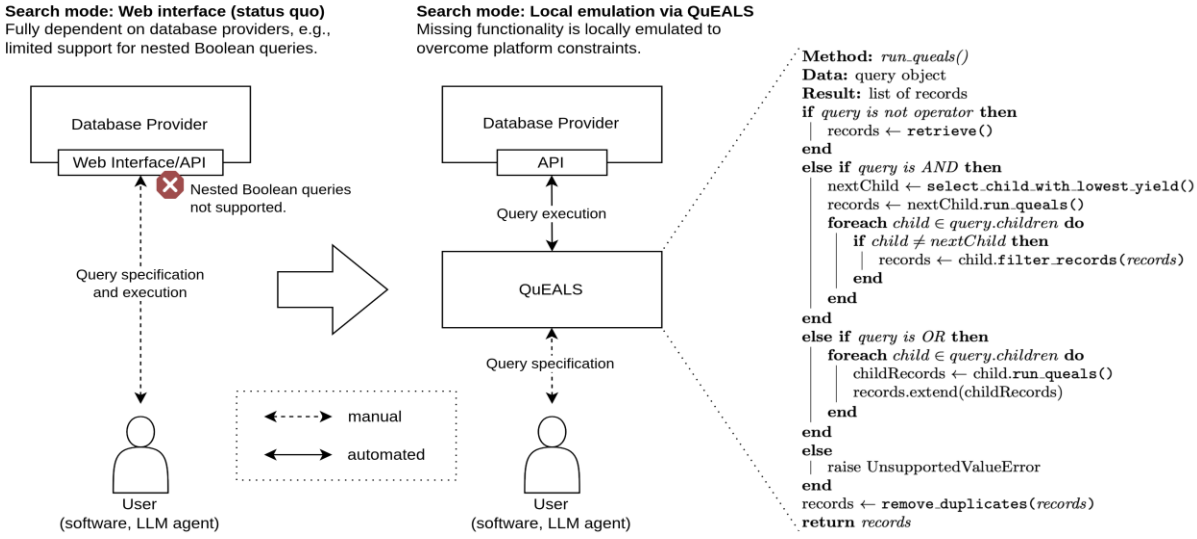


Figure 1: Architecture of QuEALS

The QuEALS algorithm details how the artifact processes a search query. The input query is represented as a tree data structure, where operators correspond to the nodes and search terms to the leaves. The algorithm processes this query tree recursively until all search terms have been evaluated. When processing a node, the algorithm distinguishes between search terms, *OR* operators, and *AND* operators. Firstly, when encountering a search term, the records for that term are retrieved from the API with a single keyword search. Secondly, when encountering an *OR* query, the method is applied recursively to all children of the query and the retrieved result sets are joined. Thirdly, when encountering an *AND* query, only the records for the child with the lowest estimated yield are retrieved to improve efficiency. This technique preserves the correctness of the result set while requiring fewer API calls, since in a Boolean *AND* query, the result set of the whole query has at most the elements of the result set of the child with the lowest yield. The other children of the *AND* query are then applied to the result set one by one until only those records remain that match the entire *AND* query.

To estimate yields, the yields of individual search terms are first collected and then used to calculate estimates for compound queries. At this stage, the calculation only has access to the yields of the search terms, but not any information about the records in the result sets. Therefore, it does not account for duplicate records when combining the yields of search terms, nor whether the exact term is included in the title of the results. As a result, the calculation is only an estimate. The formula to estimate the yield of a query is depicted below:

$$\text{estimate_yield}(\text{query}) = \begin{cases} \min(\text{estimate_yield}(c) \mid c \in \text{query.children}), & \text{if query is AND,} \\ \text{sum}(\text{estimate_yield}(c) \mid c \in \text{query.children}), & \text{if query is OR,} \\ \text{get_len_total}(), & \text{otherwise.} \end{cases}$$

Yield estimation is an additional step that increases the efficiency of the QuEALS technique; however, it is not critical for its effectiveness. As long as the API of the database provider supports a single keyword search, the technique can still be applied.

Figure 2 visualizes the process described above on a simple *AND* query. Since the algorithm is defined recursively, it can process arbitrarily complex nested queries.

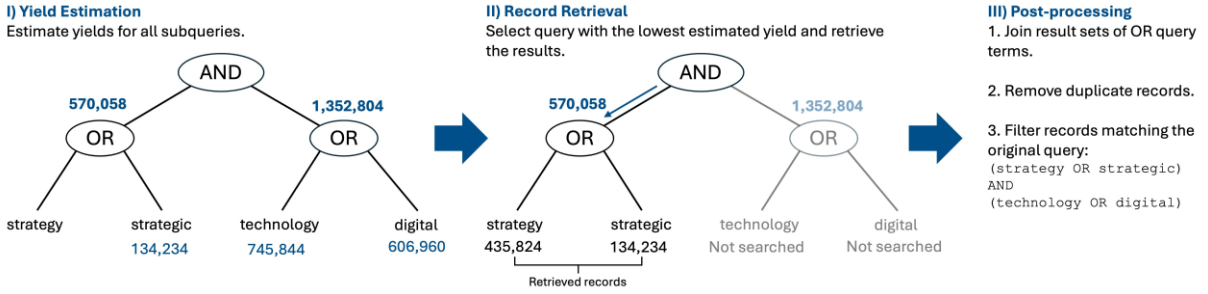


Figure 2: QuEALS process example

Development

We developed the prototype for the *Crossref* search system based on the design framework described in the previous section. The prototype was developed as part of an open-source Python package for enhancing search queries (Eckhardt et al., 2026). The complete code and documentation for the prototype are available in the Git repository.

The prototype was implemented to be client-centered and modular, as it is intended to run independently and be easily placed as an intermediary component between search system and user. Queries are implemented as a tree data structure and can be defined programmatically by the user. To implement the QuEALS algorithm, we implemented several methods for filtering records, removing duplicates and estimating yields, along with unit tests to verify correct behavior. For yield estimation we used the *Crossref* API feature to retrieve the total number of matches for a keyword search. The *search-query* library (Eckhardt et al., 2026) was used to retrieve records from the *Crossref* API via single keyword searches.

Artifact Evaluation

In the following, we evaluate the retrieval completeness, correctness, and performance of the QuEALS prototype. We selected four realistic example queries, as reported in Table 1: queries that follow the common Building-Blocks structure and queries with slight deviations from it. The results of the evaluation and the search strings are summarized in Table 2 (further details are available in the digital appendix). For each query, the final records are saved in a separate folder of the QuEALS evaluation repository.

Query	Query String
1	(strategy OR strategic) AND (technology OR digital)
2	((“knowledge work” OR “digital labor” OR “services”) AND (“platform” OR “market” OR “outsourcing”)) OR “microsourcing”
2.1	((“knowledge work” OR “digital labor” OR “services” OR “service”) AND (“platform” OR “platforms” OR “market” OR “markets” OR “outsourcing”)) OR “microsourcing”
3	(“Artificial Intelligence” OR “substitution” OR “augmentation” OR “assemblage”) AND (“ethics” OR “transparency” OR “accountability” OR “privacy” OR “maleficence” OR “justice” OR “fairness” OR “beneficence” OR “sustainability” OR “responsibility” OR “autonomy”) AND (“management” OR “strategy” OR “corporate strategy” OR “business strategy” OR “functional strategy” OR “planning processes” OR “strategic processes” OR “tactical processes” OR “operational processes”)
4	((“knowledge work” OR “digital labor” OR “services”) AND ((“platform” AND “market”) OR “outsourcing”)) OR “microsourcing”

Table 1: Test queries for the evaluation (*the queries use “Title” as the search field*)

Query 1 exemplifies the Building-Blocks approach, as different concepts (strategy, technology) are linked with the *AND* operator and different synonyms (strategy, strategic) are connected by the *OR* operator. Query 2 is based on Wagner et al. (2021) to test a slightly different structure, and assess if the prototype works for more complex structured queries. Whereas many search systems have constraints regarding Boolean operators and search strings (Gusenbauer & Haddaway, 2020), the QuEALS prototype should also work for long search queries and multiple nested Boolean operators. Query 3, which was taken from Heyder et al. (2023), reflects a more complex scenario by using 23 Boolean operators in one query. The last query depicts a slight variation of Query 2, which was developed to evaluate the correct behavior of multiple nested Boolean operators and shows the effects on performance and the number of final results. Across the test queries, we evaluated correctness, retrieval completeness, and performance, as described in the methods section.

Query	1	2	2.1	3	4
Correctness					
Query results	11,909	7,159	-	84	896
False positives	0	0	-	0	0
Retrieval completeness					
Query results[†]	26	9	9	-	4
Remaining records after filtering	23	5	9	-	4
False negatives	0	0	0	-	0
Performance					
Query results	11,909	7,159	-	84	896
Processing time (min:sec)	40:00	36:00	-	52:00	15:00

Table 2: Evaluation results

Notes. Queries: see Table 1. [†]Search space restricted to papers published in JSIS. “-” indicates that the evaluation was not applicable for that query.

Correctness Evaluation

To evaluate correctness, the search queries defined above were executed by the prototype. For each set of results, a local selection method (see appendix) was used to verify that all records retrieved matched the original query. No false positives were detected in queries 1–4.

Retrieval Completeness Evaluation

To evaluate retrieval completeness, the journal-based evaluation was conducted using *JSIS* as the reference scope. For each query, two searches were performed, expecting identical results: one using the prototype and the other using the *ScienceDirect* web interface. The search scope was limited to *JSIS* and the title field for both searches. To allow for a fair comparison of the result sets, results of the *ScienceDirect* search were corrected based on two criteria:

1. **Records must match the original query**. This was necessary to correct for expanded query interpretation by search systems because *ScienceDirect* includes term variations, such as the plural form of search terms, potentially returning results that do not strictly match the original query. Consequently, records that do not exactly match the query are excluded.
2. **Record metadata must be identical**. This was necessary to correct for the database ground truth, because in some cases, the titles of records indexed in *Crossref* do not match the titles, as indexed in *ScienceDirect* (e.g., a subtitle being included in the *JSIS* title field but not in the *Crossref* title field). Records where the title in *Crossref* did not match the title in *ScienceDirect* were excluded.

We evaluated the retrieval completeness of Queries 1, 2, 2.1, and 4. For each query, the records removed according to the two criteria are in the appendix. Query 2.1 was included in the retrieval completeness evaluation as a variant of Query 2, with term variations added to mimic the expanded query interpretation by *ScienceDirect*.

- **Query 1**: *ScienceDirect* returned 26 results, and QuEALS returned 23 results. From the *ScienceDirect* set, we removed 2 records, which did not match the original query (criterion 1). In addition, we removed 1 record from *ScienceDirect*, which resulted from different titles (criterion 2). A comparison showed that the remaining records of *ScienceDirect* and QuEALS are identical, confirming retrieval completeness.
- **Query 2**: *ScienceDirect* returned 9 results, and QuEALS returned 5 results. From the *ScienceDirect* set, we removed 4 records, which did not match the original query (criterion 1). A comparison showed that the remaining records of *ScienceDirect* and QuEALS are identical, confirming retrieval completeness.
- **Query 2.1**: *ScienceDirect* and QuEALS each returned 9 results. A comparison showed that the records in the result sets are identical, confirming retrieval completeness.
- **Query 3**: not evaluated, as it exceeds the 150-character limit of the *ScienceDirect* title search.
- **Query 4**: *ScienceDirect* and QuEALS each returned 4 results. A comparison showed that the records in the result sets are identical, confirming retrieval completeness.

Performance Evaluation

To evaluate performance, we measured the processing time of the prototype in different scenarios, i.e., the time it takes from the start of the QuEALS prototype until the results are returned. The evaluation table shows the final number of records matching the query after local filtering. Note that the number of records retrieved from the API before filtering can be substantially larger than the final result set. For Query 1, it took 40 minutes from the start of the QuEALS prototype to return 11,909 records (5.0 records/second). Query 2 was processed in 36 minutes, returning 7,159 records (3.3 records/second); Query 3 returned 84 records after 52 minutes (0.03 records/second); Query 4 returned 896 records after 15 minutes (1.0 records/second).

Scalability and Performance Optimization

With regard to the runtime and scalability of QuEALS, API rate limits and request-response times are the primary bottlenecks, whereas local filtering and deduplication make only a minor contribution to runtime. Accordingly, scalability is primarily determined by the number of records that must be retrieved from the API. Since the API rate limit and per-request retrieval limit are fixed, retrieval time scales linearly with the retrieved count. For a typical building-block AND query, the retrieved count is determined by the child with the lowest estimated yield. If this child is an OR query, its yield can grow quickly with the number of search terms because each term requires a separate keyword search. Query 3 exemplifies such a worst-case scenario in which even the child with the lowest estimated yield is an OR query containing a large amount of search terms. Although Query 3 produces a small result set after local filtering, the disproportionately large count of records the prototype must retrieve results in a relatively long runtime. In contrast, for Query 1 with the scope restricted to the journals *MIS Quarterly* and *Information Systems Research*, total execution time decreased from 40 minutes to 20 seconds due to the lower retrieval volume resulting from the narrower search scope.

A key question for the scalability of QuEALS is therefore how the number of records retrieved before local filtering can be reduced in realistic search scenarios. One effective approach is to specify filters directly in the API requests, narrowing the scope of the search before retrieval. In practice, searches for systematic reviews are often scoped with additional filters. Journal filters, for example, allow researchers to limit their scope to highly valued results and, therefore, only retrieve relevant results for their search (Webster & Watson, 2002). Similarly, time-based filters are often used in the context of search updates and living reviews, in which literature samples are periodically updated with newly published records. In Table 3, we used the yield estimation feature to estimate the yield of real-world IS search queries under different journal scopes, and compared them to unfiltered (global) search. The journal scopes were selected from *LitBaskets* (Wang et al., 2026), with each successive basket extending the previous one by including additional journals. The 2XS basket corresponds to the *AIS Basket of Eight*. The results show substantial reductions in the estimated yields and, consequently, the number of records the prototype would need to retrieve across all journal-based searches, even for larger baskets containing up to 51 peer-reviewed IS journals (basket M). Furthermore, combining journal scopes with a 2025 publication year filter further greatly reduces retrieval volume.

Publication years	Journal basket	Q5	Q6	Q7	Q8	Q9	Q10
All years	2XS	31	128	179	265	1,157	1,221
All years	XS	68	312	682	576	3,273	5,349
All years	S	128	584	1,285	1,138	6,993	8,947
All years	M	281	964	2,741	1,879	9,807	14,755
All years	Global	122,616	413,582	417,487	476,498	1,037,590	4,732,965
2025	2XS	0	3	3	5	31	21
2025	XS	1	19	47	12	179	186
2025	S	3	40	77	28	386	306
2025	M	8	57	171	55	536	495
2025	Global	14,386	35,216	46,997	25,120	96,163	331,487

Table 3: Estimated yield by journal filtering strategy

Notes. Full queries are available in the digital appendix.

The analysis indicates that runtimes for realistically scoped searches, particularly those with journal filters, are more feasible than those observed for broad, global searches. In practice, particularly when runtime is a critical consideration, the prototype is better suited to more narrowly scoped searches that utilize filters.

In the context of systematic reviews, a constrained search scope with filters might also represent a more realistic search scope compared to an unrestricted *Crossref* search, since systematic searches typically are designed to achieve a suitable balance of precision and recall.

Discussion

Principal Findings

Overall, the evaluation provides evidence of feasibility and promising performance in the examined scenarios. No false positives were observed in the correctness evaluation, and no false negatives were observed in the evaluated retrieval-completeness cases. The performance evaluation showed that even for complex, nested queries the prototype returned the results within an acceptable runtime, with all tests completing in under an hour. Furthermore, applying search filters with the prototype can both narrow the search scope and significantly reduce retrieval volume and consequently, runtime.

Contributions

With reference to the research questions, we demonstrated that QuEALS can facilitate the automation of academic literature searches even if APIs do not offer support for nested Boolean queries. It provides the possibility to retrieve literature from APIs of search systems and reduces manual operations that are often prone to errors, such as adapting search strings to the syntax requirements of each web interface. The evaluation provides initial evidence that QuEALS is feasible for the examined *Crossref* search scenarios and that filters can substantially improve its runtime performance. Through the automated literature retrieval, researchers can experience cost and time savings, especially for search systems that have substantial limitations, such as *Crossref*.

The QuEALS technique enables searching *Crossref* with nested Boolean queries, which was previously not possible through either the web interface or with the API. QuEALS's filtering methods also enable searches for open compound words like "big data analytics", which is currently not supported by *Crossref*. With its generalized query format, independent emulators like QuEALS could provide a common query-processing layer across multiple search systems, while source-specific adapters would remain responsible for retrieving records from the respective APIs. Finally, the QuEALS prototype demonstrates how essential search capabilities can be developed independently without relying on the cooperation of search system providers.

Limitations and Future Work

A limitation of the emulator approach is that QuEALS can address selected shortcomings of search systems, while others remain dependent on the providers (see Al-Zubidy & Carver, 2019; Krüger et al., 2020). For example, inconsistent retrieval behavior and API availability cannot be addressed by QuEALS. Furthermore, temporary interruptions on the server side or in the local network can disrupt long-running retrieval operations, while API rate limits can reduce performance for large retrievals. As a result, increasing the robustness of such operations is an important area of ongoing work. Rather than processing a large retrieval as a single operation, QuEALS could partition searches into smaller batches, for example by publication year, and persist retrieved records and completion information for each batch. This would allow interrupted searches to resume from the last incomplete batch instead of restarting the entire retrieval process. We further plan to complement this mechanism with completeness verification by recording the expected and actually retrieved number of records and marking batches as completed only after successful verification.

The evaluation is limited by the relatively small number of test queries and its primary focus on *Crossref*. In addition, retrieval completeness was assessed within a restricted setting (comparison with JSIS on *ScienceDirect*), meaning that the results provide evidence of feasibility and promising performance rather than comprehensive validation across academic search contexts. Future evaluations should therefore examine a broader range of realistic query structures, larger retrieval volumes, and additional search systems and APIs.

Finally, the initial functionality of QuEALS focuses on emulating the Boolean operators *AND* and *OR*, is restricted to title searches, and relies on exact-term matching. Wildcard-based expansions would need to

be specified explicitly in the search query. Accordingly, extending this functionality represents another important direction for future work. This includes support for additional operators such as *NOT* and proximity operators (*NEAR*, *WITHIN*), further search fields and filters, and wildcard-based term expansion. QuEALS could also be extended to additional search systems through source-specific adapters, while retaining its generalized query representation and local query processing. This could standardize selected query capabilities across multiple search sources, although differences in metadata, searchable fields, and API behavior would remain. Thereby, selected limitations and missing query capabilities of search systems (Gusenbauer & Haddaway, 2020; Li & Rainer, 2022) could be compensated for through independent emulation rather than requiring corresponding functionality to be implemented by the search-system providers.

Conclusion

QuEALS enables researchers to overcome limited API query capabilities when conducting automated literature searches. Its distinctive contribution is an independent query-processing layer that implements selected search capabilities outside the underlying search system. This challenges the assumption that advanced query capabilities must be implemented by search-system providers. Instead, if an API supports sufficiently reliable and consistent keyword retrieval, QuEALS can provide additional query-processing functionality on top of comparatively limited native search capabilities. We demonstrate this approach with a prototype for the *Crossref* API that supports nested queries with *AND* and *OR* operators. Additional capabilities, such as proximity operators or wildcards, could be implemented in future work. The evaluation provides evidence of feasibility and promising performance for the examined scenarios. Scalability remains a concern for retrievals spanning the global *Crossref* corpus, whereas more realistic, discipline-bounded search scopes substantially reduce retrieval volumes and runtime.

The modular and programmatic design of QuEALS allows its query-processing capabilities to be reused across different literature review tools and workflows. As an open-source artifact, it can be used directly by researchers, integrated into literature review software, and adapted to specific research needs. This reuse is particularly relevant for living reviews (Elliott et al., 2017), where established queries can be rerun with time-based filters that restrict retrieval to records published since the previous search. More broadly, the same independent query-processing approach could be extended across multiple search systems. Source-specific adapters could be designed to handle retrieval and API-specific requirements for individual search systems. In the context of AI-augmented and agentic workflows, QuEALS provides a deterministic execution component for specified search queries. While agents may support query formulation, refinement, and workflow orchestration, specified search queries can be delegated to QuEALS for deterministic and reproducible execution. QuEALS and agentic approaches are therefore complementary rather than competing.

Digital Appendix

The evaluations are available on GitHub: https://github.com/fs-ise/beals-crossref-evaluation.

References

Al-Zubidy, A., & Carver, J. C. (2019). Identification and prioritization of SLR search tool requirements: An SLR and a survey. Empirical Software Engineering, 24(1), 139–169. https://doi.org/10.1007/S10664-018-9626-5

Booth, A. (2008). Unpacking your literature search toolbox: On search styles and tactics. Health Information & Libraries Journal, 25(4), 313–317. https://doi.org/10.1111/J.1471-1842.2008.00825.X

Cooper, C., Booth, A., Varley-Campbell, J., Britten, N., & Garside, R. (2018). Defining the process to literature searching in systematic reviews: A literature review of guidance and supporting studies. BMC Medical Research Methodology, 18(1), 85. https://doi.org/10.1186/s12874-018-0545-3

Eckhardt, P., Ernst, K. M., Fleischmann, T., Geßler, A., Schnickmann, K., Thurner, L., & Wagner, G. (2026). Search-query: An open source python library for academic search queries. Journal of Open Source Software, 11(118), 8775. https://doi.org/10.21105/JOSS.08775

Elliott, J. H., Synnot, A., Turner, T., Simmonds, M., Akl, E. A., McDonald, S., Salanti, G., Meerpohl, J., MacLehose, H., Hilton, J., et al. (2017). Living systematic review: 1. Introduction—the why, what, when, and how. Journal of Clinical Epidemiology, 91, 23–30. https://doi.org/10.1016/j.jclinepi.2017.08.010

Gonzalez-Toral, S., Freire, R., Gualan, R., & Saquicela, V. (2019). A ranking-based approach for supporting the initial selection of primary studies in a Systematic Literature Review. 2019 XLV Latin American Computing Conference (CLEI), 1–10. https://doi.org/10.1109/CLEI47609.2019.235079

Gusenbauer, M., & Gauster, S. P. (2025). How to search for literature in systematic reviews and meta-analyses: A comprehensive step-by-step guide. Technological Forecasting and Social Change, 212, 123833. https://doi.org/10.1016/J.TECHFORE.2024.123833

Gusenbauer, M., & Haddaway, N. R. (2020). Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources. Research Synthesis Methods, 11(2), 181–217. https://doi.org/10.1002/JRSM.1378

Gusenbauer, M., & Haddaway, N. R. (2021). What every researcher should know about searching – clarified concepts, search advice, and an agenda to improve finding in academia. Research Synthesis Methods, 12(2), 136–147. https://doi.org/10.1002/JRSM.1457

Haddaway, N. R., Rethlefsen, M. L., Davies, M., Glanville, J., McGowan, B., Nyhan, K., & Young, S. (2022). A suggested data structure for transparent and repeatable reporting of bibliographic searching. Campbell Systematic Reviews, 18(4), e1288. https://doi.org/10.1002/CL2.1288

Hevner, A. R., March, R., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75. https://doi.org/10.2307/25148625

Heyder, T., Passlack, N., & Posegga, O. (2023). Ethical management of human-AI interaction: Theory development review. The Journal of Strategic Information Systems, 32(3), 101772. https://doi.org/10.1016/J.JSIS.2023.101772

Hiebl, M. R. W. (2023). Sample Selection in Systematic Literature Reviews of Management Research. Organizational Research Methods, 26(2), 229–261. https://doi.org/10.1177/1094428120986851

Krüger, J., Lausberger, C., von Nostitz-Wallwitz, I., Saake, G., & Leich, T. (2020). Search. Review. Repeat? An empirical study of threats to replicating SLR searches. Empirical Software Engineering, 25(1), 627–677. https://doi.org/10.1007/S10664-019-09763-0

Larsen, K. R., Hovorka, D., Dennis, A., & West, J. (2019). Understanding the Elephant: The Discourse Approach to Boundary Identification and Corpus Construction for Theory Review Articles. Journal of the Association for Information Systems, 20(7). https://doi.org/10.17705/1JAIS.00556

Li, Z., & Rainer, A. (2022). Academic search engines: Constraints, bugs, and recommendations. Proceedings of the 13th International Workshop on Automating Test Case Design, Selection and Evaluation, 25–32. https://doi.org/10.1145/3548659.3561310

MacDonald, H., Comer, C., Foster, M., Labelle, P. R., Marsalis, S., Nyhan, K., Premji, Z., Rogers, M., Splenda, R., Stansfield, C., & Young, S. (2024). Searching for studies: A guide to information retrieval for Campbell systematic reviews. Campbell Systematic Reviews, 20(3), e1433. https://doi.org/10.1002/cl2.1433

MacFarlane, A., Russell-Rose, T., & Shokraneh, F. (2022). Search strategy formulation for systematic reviews: Issues, challenges and opportunities. Intelligent Systems with Applications, 15, 200091. https://doi.org/10.1016/J.ISWA.2022.200091

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Prester, J., Wagner, G., Schryen, G., & Hassan, N. R. (2021). Classifying the ideational impact of Information Systems review articles: A content-enriched deep learning approach. Decision Support Systems, 140, 113432. https://doi.org/10.1016/J.DSS.2020.113432

Shakeel, Y., Krüger, J., Von Nostitz-Wallwitz, I., Lausberger, C., Durand, G. C., Saake, G., & Leich, T. (2018). (Automated) literature analysis: Threats and experiences. Proceedings of the International Workshop on Software Engineering for Science, 20–27. https://doi.org/10.1145/3194747.3194748

Velez-Estevez, A., Perez, I. J., García-Sánchez, P., Moral-Munoz, J. A., & Cobo, M. J. (2023). New trends in bibliometric APIs: A comparative analysis. Information Processing & Management, 60(4). https://doi.org/10.1016/J.IPM.2023.103385

Wagner, G., Lukyanenko, R., & Paré, G. (2022). Artificial intelligence and the conduct of literature reviews. Journal of Information Technology, 37(2), 209–226. https://doi.org/10.1177/02683962211048201

Wagner, G., & Prester, J. (2024). CoLRev - an open-source environment for collaborative literature reviews. https://github.com/CoLRev-Environment/colrev. https://doi.org/10.5281/ZENODO.11668338

Wagner, G., Prester, J., Mousavi, R., Lukyanenko, R., & Paré, G. (2026). Generative artificial intelligence for literature reviews. Journal of Information Technology, 41(4), 148–170. https://doi.org/10.1177/02683962261425675

Wagner, G., Prester, J., & Paré, G. (2021). Exploring the boundaries and processes of digital platforms for knowledge work: A review of information systems research. The Journal of Strategic Information Systems, 2021 Review Issue, 30(4), 101694. https://doi.org/10.1016/J.JSIS.2021.101694

Wang, B., Boell, S. K., & Ciriello, R. F. (2026). LitBaskets: Supporting exploratory literature searches for the information systems research community. Communications of the Association for Information Systems, 58(1), 56.

Webster, J., & Watson, R. T. (2002). Analyzing the Past to Prepare for the Future: Writing a Literature Review. MIS Quarterly, 26(2), xiii–xxiii. https://www.jstor.org/stable/4132319

Zwanenburg, S. P. (2023). PermuSearch V1.0 [Computer software]. https://permusearch.com